GROUNDED DI / EXTERNAL CONTROL AUDITS

GoDaddy DNS Control Audit

THE FALLEN WARRIORS — PUBLIC RESEARCH EDITION

October 2026 | Grounded DI LLC

Dedicated to Mark S. Weinstein’s brother-in-law.

Bounded local DNS reconciliation tests. Strong passing controls. One final-state verification gap. No verified production security vulnerability.

READING GUIDE

How to read this research edition

This is an independent review of a supplied October 22, 2026 research evidence package. It is not sponsored by, endorsed by, or certified by GoDaddy.

SECTION RANGE	CONTENT
01–04	Executive assessment, evidence boundaries, source identity and custody
05–10	DNS control model, test baseline, local replays and findings
11–14	Passing controls and separate synthetic DI² sidecar
15–19	Adversarial self-audit, hardening, exclusions and public-safety review
20–21	References, findings and release-verification statement

Reading the evidence labels

VERIFIED means the supplied record contains directly inspectable supporting evidence or a confirmed public source. It does not mean this edition re-ran historical tests. INFERENCE identifies supported implications. UNVERIFIED and UNTESTED mark conditions not established.

The public edition deliberately omits raw code, internal runner detail and proprietary CodeCraft material. The attached verification record distinguishes newly performed package checks from historical test claims.

01 / ASSESSMENT

Executive summary

At the pinned October 2026 GoDaddy CLI revision, a focused local replay identified two circumstances in which the observed final DNS record set differed from the requested exact set. The first was an explicitly reported create-then-cleanup partial failure. The second was a concurrent insertion after the read that was absent from the stale write plan. The second is the sharper final-state verification gap.

Neither observation establishes an exploitable vulnerability or production GoDaddy behavior. The original evidence describes synthetic DNS records, a loopback HTTP server, and calls into pinned production DNS functions. The tests did not authenticate to GoDaddy, change a real DNS zone, or exercise its production authorization controls.

The favorable findings matter: retries after an ambiguous successful create converged in the fixture without a second create, completed repeated invocations did not repeat a mutation, and all recorded requests remained within the requested example.test zone.

Dimension	Disposition
Final classification	Bounded consistency/control-hardening observation
Security finding	None established
Native tests	1,223 reported passing; not rerun for this book
Focused replay	Two byte-identical historical observation files; evidence reviewed
Production DNS	Untested

02 / CLAIM BOUNDARIES

What this publication does and does not prove

This edition independently checks the integrity and contents of the supplied evidence archive. It does not re-execute the October 22 Rust replay, reinstall the historical toolchain, run the full DI² harness, or reproduce effects against GoDaddy production infrastructure.

Accordingly, “verified locally” describes the original report’s recorded tests, corroborated by the associated observation JSON and hashes. It does not describe fresh testing performed for this edition. The two round files are equal; identical files alone do not prove independent runs occurred without trusted external execution attestations.

The strongest conclusion is contract-specific: the tested read-plan-write path can finish its planned actions without independently checking whether a concurrent writer left the DNS record set unequal to the desired exact values.

Label	Interpretation
VERIFIED in supplied evidence	Direct records/hashes support reported fixture outcomes
INFERENCE	Relevance of a final GET to stronger postconditions
UNVERIFIED	Production frequency, user impact, undocumented API preconditions
UNTESTED	Real domains, account scope, propagation, active service

03 / TARGET

Source identity and pinning

Target repository: github.com/godaddy/cli. The audit records commit 6aa64b60103d6c0807a3da9e4cde9380381e29a1, corresponding to release 0.2.31 on October 22 UTC. A present-day GitHub connector lookup confirmed that this commit exists and records tree 6f60de948eb8f89ae90b00c6a813e731354ed8fd.

The primary entry is gddy dns set. The report also references gddy dns add and gddy dns delete where they share write-path behavior. Source-path hashes in the historical audit distinguish the inspected implementation; the original source checkout is not bundled in this public book.

The package contains historical source-path digests and replay code, but this publication relies on descriptions and evidence mapping, not reproduction of complete third-party implementation files.

Object	Recorded identity
Pinned commit	6aa64b60103d6c0807a3da9e4cde9380381e29a1
Git tree	6f60de948eb8f89ae90b00c6a813e731354ed8fd
Primary path	rust/src/dns/set/mod.rs
Planning path	rust/src/dns/set/plan.rs
Write path	rust/src/dns/set/write.rs
Outcome path	rust/src/dns/set/outcome.rs

04 / SOURCE PACKAGE

Evidence inventory and custody

The original outer ZIP contains three nested archives, an index, a short README, and checksums. Extraction was confined to the working area. The originals were not modified. Nested member names were checked for traversal paths, and ZIP CRC checks succeeded.

The October 22 DNS archive is the principal technical evidence. Its internal checksum list contains the Markdown report, observation rounds, harness summary, integrity record, reproduction notes and source snippets. All listed hashes matched the supplied member bytes in this publication pass.

The CodeCraft v2 and v3 archives are separate Grounded DI products and do not establish a GoDaddy defect. Their content has been excluded from the public edition.

Item	Integrity result
Outer collection	e35a8f7b0caf3ecc854a9b648de99bee63f5aee0c6f8f3a27baeee04e99963ca
DNS evidence ZIP	74a2a6bceb265d240a1dd830fe4b8797b8b66e3b9fab7a472a1a020fe7a38438
Native rounds	Equal bytes; SHA-256 4845769e5589...13db3f2bb5
Sidecar rounds	Equal bytes; SHA-256 a2b2dd840ba8...f1929f6a
Internal checksums	11 / 11 passed

05 / SYSTEM MODEL

How the tested DNS operation works

The audited path reads existing DNS records for a selected zone, name and type. A local planner compares those records with requested values. The program then dispatches create and delete operations to implement its plan and summarizes action outcomes.

The characteristic replacement order is create-before-delete. That protects the old value when the new value cannot be created, but means a failed cleanup can temporarily or persistently leave both values. The source report does not identify a transaction or final exact-set read at the end of the observed handler path.

The design question is not whether every intermediate state equals the desired state. It is what the CLI’s successful completion should certify after a concurrent change, and how it should communicate uncertainty when that postcondition was never checked.

Stage	Behavior
READ	Fetch current DNS records for zone/name/type
PLAN	Determine retain, create, replace, surplus-delete actions
WRITE	Execute individually dispatched record operations
REPORT	Summarize planned actions and failures
MISSING IN TESTED PATH	Independent final read to certify the requested exact set

06 / HISTORICAL TEST RESULTS

Native test lane: strong passing baseline

The original report records a clean Rust-native lane: cargo check, Clippy with warnings denied, formatting, module-size checks, and focused DNS test groups all passed. Its complete offline workspace test command reported 1,223 unit tests passing, zero failures, and 49 generated documentation tests ignored.

These results support a substantial existing test baseline. They do not prove that all concurrency states are covered or that the focused synthetic loopback reproduces production service behavior.

The present publication review read the historical command/result record but did not run Cargo or recalculate those test counts in a fresh source checkout.

Check	Reported outcome
cargo check --workspace	PASS — historically reported
cargo clippy --workspace -- -D warnings	PASS — historically reported
cargo fmt --check	PASS — historically reported
Focused dns::set tests	24 PASS — historically reported
Focused dns:: tests	58 PASS — historically reported
Workspace unit tests	1,223 PASS; 0 failures — historically reported

07 / TEST DESIGN

Local replay contract and instrumentation

The historical focused Rust replay was inserted as a test-only module in an isolated worktree. It drove the pinned production functions fetch_records, plan_set, write_with_conflict_handling and apply_replace through a stateful loopback HTTP server.

For each fixture, the observation files recorded the initial state, requested state, HTTP method and path, POST body, DELETE target, failure types and final synthetic zone records. The server represented deterministic fixture behavior, not GoDaddy cloud infrastructure or its authorization policy.

The documentation-only example.test domain and addresses in the TEST-NET IPv4 ranges minimize accidental confusion with live customer data. Both supplied native observation rounds have identical hashes, supporting internal result consistency.

Field	Boundary / value
Endpoint	Local loopback; synthetic HTTP
Domain	example.test
Old A value	192.0.2.1
Desired A value	192.0.2.10
Concurrent external fixture value	198.51.100.77
Original source checkout	Recorded as clean after test

08 / PARTIAL UPDATE

Finding A: cleanup failure after creation

The fixture began with www A pointing to 192.0.2.1. The requested replacement was 192.0.2.10. The replay observed GET, POST for the desired value, and DELETE for the old record returning a synthetic HTTP 500 error.

The final synthetic zone contained both 192.0.2.1 and 192.0.2.10. The command reported a cleanup failure. The historical report correctly treats this as an expected consequence of non-atomic create-before-delete replacement, not as silent success.

Engineering implication: this is an operational consistency and recovery concern for callers who require an exact set. The explicit error is a material control; the observed mismatch should not be represented as concealed or as an unauthorized security action.

Event	Observed fixture value
Initial set	[192.0.2.1]
Desired set	[192.0.2.10]
Response sequence	GET → POST success → DELETE synthetic 500
Final set	[192.0.2.1, 192.0.2.10]
Reported failure kind	cleanup

09 / STALE SNAPSHOT

Finding B: concurrent insertion after planning

After the initial filtered GET and plan construction, the fixture inserted an additional same-name/type A record with value 198.51.100.77. The command then followed its precomputed plan, created 192.0.2.10 and deleted old-1.

Every planned core action returned success in the observation, but the final fixture state contained both the newly inserted value and the requested value. No final GET established exact-set convergence before the handler summarized the planned actions.

The external record was introduced by the fixture, not by the CLI. The observation supports a stale-snapshot verification gap; it does not show malicious external access or prove how frequently concurrent edits happen in production.

Stage	Observation
Requested exact set	[192.0.2.10]
Concurrent fixture insertion	198.51.100.77 after the read
Planned mutations	Create desired value; delete old-1
Final observed set	[198.51.100.77, 192.0.2.10]
Core action outcomes	Successful; no final-state re-read

10 / POSTCONDITION

Why the concurrency finding matters

A read-plan-write command has two separate success meanings: all planned operations completed, and the shared resource now has the exact state originally requested. The tested sequence demonstrated that the first can hold while the second is false.

Without a final read, the command cannot directly certify that a concurrent writer did not alter the same scope after the initial snapshot. A verification read also has its own race: a subsequent writer could modify the zone immediately after it. Thus the sensible control is a clearer postcondition or a versioned conditional-write facility, not an unsupported promise of global atomicity.

No evidence in the supplied material establishes that a GoDaddy production API exposes ETag or version-precondition writes for this path. The report appropriately leaves that design alternative conditional.

Concept	Distinction
Action-level success	The planned POST and DELETE operations returned success
Exact-set convergence	Final record set equals the requested set
Observed gap	Action success without independent final-set check
Bounded remediation	Final read/compare, explicit mismatch result, cautious replan

11 / RESILIENCE

Passing control: lost response, fresh retry

The fixture committed a create operation and then closed the response stream, leaving the first invocation uncertain about the result. The user-initiated fresh retry re-read the state and did not send a second POST. It removed the old record and reached the desired set.

The recorded total POST count was one. This is a favorable counterexample to an assertion that the path necessarily duplicates records after an ambiguous response. It supports convergence in this specific fixture, not universal exactly-once semantics or a server-side idempotency guarantee.

A crucial difference is retrying with a fresh read rather than replaying the original mutation request blindly.

Stage	Recorded behavior
First POST	Desired value committed; response lost
First invocation	Failure reported
Second invocation	Fresh GET; no second POST
Total POSTs	1
Final set	[192.0.2.10]

12 / IDEMPOTENT-LIKE CASE

Passing control: repeat invocation

In the completed-request fixture, the first invocation performed GET, POST and DELETE. The second invocation performed only a GET. No further write or durable duplicate effect was observed.

This is a concrete passing control that narrows any critique of the CLI. It demonstrates a read-only repeat under the tested state and record type. It does not prove every operation is idempotent across unexpected API responses, all record types, or concurrent writes.

The better technical description is “no additional mutation on the tested repeated invocation,” rather than a categorical guarantee about the entire DNS product.

Event	Observation
First invocation	GET / POST / DELETE
Second invocation	GET only
Second write count	0
Result class	Local passing control

13 / SAFETY BOUNDARY

Passing control: scope isolation

The native observation records show that the replayed requests stayed within the synthetic requested zone path: /v3/domains/zones/example.test/dns-records. No request to another domain or zone appeared in the four fixtures.

The report also describes early rejection when an initially read record lacks a recordId, and consistent reuse of the captured domain argument across client operations. These are favorable control properties in the inspected source path.

This test does not assess server-side authorization, tenant isolation, ownership resolution or production account behavior. It would be an overstatement to translate a synthetic same-domain request log into a general claim about GoDaddy cloud security.

Dimension	Scope
Synthetic domain	example.test
Unexpected zone requests	0 in recorded fixtures
Production account boundaries	Untested
Production DNS propagation	Untested

14 / HARNESS BOUNDARY

Separate DI² sidecar evidence

The DNS-shaped DI² sidecar is a separate synthetic authorization exercise. It is not evidence that GoDaddy implements, depends upon, or was evaluated against DI² controls in production.

The supplied sidecar records passing synthetic checks for immutable approved-plan copying, altered approval-binding denial, duplicate-event suppression and expiration at the final invocation boundary. Its two sidecar rounds are byte-identical.

The original evidence also reports two full-harness rounds: 240 registered checks, 105 core and 135 unit checks each round, 15/15 curated mutations detected, 8/8 expiry mutations detected, and recovery across seven injection points. The complete historical harness archive was not supplied within this three-ZIP collection, so those execution claims were not newly reproduced here.

Synthetic property	Recorded result
Plan mutation after prepare	PASS — supplied sidecar result
Altered binding	PASS — denied, zero effects
Duplicate dispatch	PASS — second denied
Expiry at invocation	PASS — denied before invocation
Model routes executed	False (historical record)

15 / CHALLENGE THE CLAIMS

Adversarial self-audit

Challenge 1: does the cleanup failure reveal a defect? Not necessarily. Separate create and delete calls are non-atomic; the command explicitly surfaced cleanup failure. The proper observation is a reported partial state, not a hidden success.

Challenge 2: is the concurrent insertion artificial? Yes, the interleaving was injected by the fixture. That limits production impact claims while preserving the validity of the tested stale-plan counterexample.

Challenge 3: do two identical observation files prove independent execution? No. They establish equal recorded bytes; a signed runner attestation or witnessed rerun would be needed to establish independent execution provenance.

Challenge 4: does 1,223 passing tests imply no gaps? No. The passing baseline and the replay counterexample address different conditions. Neither invalidates the other.

Review axis	Result
Overclaim avoided	The report does not demonstrate production exploitation
Limitation retained	No CLI subprocess/authenticated startup coverage
Strength retained	Clean successful retry and repeat controls
Correction to headline	Classify OBS-01 as reported partial update; OBS-02 as final-state gap

16 / RECOMMENDATIONS

Practical engineering hardening

The lowest-cost proposed improvement is a final filtered read after planned writes complete successfully. Compare the observed record multiset against requested values with the same content equivalence rules used in planning. When they differ, report a distinct non-zero final-state mismatch rather than silently certifying convergence.

Automatic deletion of newly observed records is not automatically safe: the concurrent change could represent a legitimate separate actor. A cautious tool would present both sets, allow an explicit replan or rely on server-supported conditional writes when such features exist.

Preserve create-before-delete ordering and the existing explicit cleanup failure report. Preserve fresh reads on user-initiated retries. These are protections supported by the positive controls, not problems to remove in pursuit of a single test passing.

Priority	Direction
Near-term	Final GET and exact-set comparison
Failure contract	Explicit concurrent-change / mismatch status
Concurrency prevention	Optional version preconditions if API supports them
Avoid	Blind deletion of concurrent records
Preserve	Create-before-delete; clear cleanup errors; fresh retries

17 / REPLAY BOUNDARY

Reproducibility and limits

The private source archive includes a standalone Rust test-only replay, integration instructions, two observation files and an independent Python sidecar. Those files are not being republished in this public edition because they belong to the retained research/replay package.

The public edition provides source pinning, test conditions, observed request order, example-state data, artifact hashes and precise claim boundaries. These make the research reviewable, but not a self-contained executable replay kit.

To reproduce the historical results independently, a future reviewer would need the specific pinned source, locked dependencies, a separately controlled fixture matching the recorded interleavings, and a fresh signed execution record. No production service should be exercised without separate authorization.

Asset / claim	Status
Original full replay archives	Private / not included
Historical commands and counts	Reported, not re-executed
Native JSON observation pair	Inspected and hashed; not published
Public book	Explanation and checksums only
Production generalization	Not supported

18 / EXCLUDED MATERIAL

CodeCraft edition comparison

The two CodeCraft DI GoDaddy ZIPs contain identical index.html, style.css, form-handler.php, README.md and Instructions_CodeCraft_DI.docx bytes. Their identically named two-page MagicPDF files differ at the binary level.

The MagicPDFs have equal extracted text and equal rendered pixels on both pages. Their PDF creation/modification timestamps differ and their stream/document identifier bytes differ. This supports a metadata/serialization change rather than an observed change to visible content; it does not claim all hidden structures were forensically proven equal.

The CodeCraft PDFs make broad product performance and IP-protection claims not tested by the DNS audit. Those claims and proprietary instructions are deliberately excluded. There is no basis to attribute the CodeCraft bundles to GoDaddy’s CLI or count them as evidence of product weaknesses.

Comparison	Finding
Non-PDF members	5 / 5 byte-identical
MagicPDF text	Equal across two pages
MagicPDF rendered pixels	Equal across two pages
Creation timestamps	Differ
Editorial decision	Exclude CodeCraft implementation from public book

19 / RELEASE REVIEW

IP, attribution and publication safety

This book paraphrases a locally supplied audit of a public repository, cites the pinned public source, and does not reproduce full upstream third-party source files, CodeCraft instructions, private harness code or raw private replay archives.

The package itself contains no intentionally included credentials, tokens, private customer data or real-domain operations. The public examples are synthetic. GoDaddy, its CLI names and upstream paths are used descriptively; Grounded DI does not claim sponsorship, endorsement or certification by GoDaddy.

This targeted review is not an opinion that every conceivable copyright, patent or contractual issue has been extinguished. The release record includes a conditional owner-review status, and the public candidate is not being published. No specific harmful exploitation sequence or unpublished credential is necessary to explain the demonstrated findings.

Category	Disposition
Vendor source	Pinned public repository; descriptive references
Private DI harness	Excluded
Proprietary CodeCraft prompts	Excluded
Private runtime paths	Excluded
Third-party long excerpts	None intentionally reproduced
Release authorization	Owner review outstanding

20 / CITATIONS

Source references and provenance

Primary local source: GoDaddy_DNS_Set_Control_Audit_2026-10-22.md inside GoDaddy_DNS_Set_Audit_Evidence_2026-10-22.zip. The main report provides the audit conditions, command list, fixture narratives and qualified disposition. Its accompanying checksum manifest and observation JSON were checked directly.

Public repository reference: https://github.com/godaddy/cli/tree/6aa64b60103d6c0807a3da9e4cde9380381e29a1. Public source path examples: rust/src/dns/set/mod.rs, plan.rs, write.rs, outcome.rs. Present-day repository lookup confirmed the commit and tree identity.

Additional private evidence: native_dns_observations_round1.json and round2.json; di2_sidecar_round1.json and round2.json; di2_full_replay_summary.json; di2_integrity.json; manifest.json. No external party or vendor has certified the audit.

Evidence ID	Reference
E-01	Local audit Markdown and source-path hashes
E-02	Native DNS JSON observations (two equal rounds)
E-03	Separate DI² sidecar JSON (two equal rounds)
E-04	Harness summary and integrity JSON — historical
E-05	Package SHA256SUMS and source ZIP hash
E-06	Public pinned GitHub commit lookup

21 / CONCLUSION

Final verdict and verification statement

Verdict: The supplied evidence supports a bounded GoDaddy CLI DNS consistency hardening observation. OBS-01 shows a visible partial update after cleanup failure; OBS-02 shows a stale-plan exact-set mismatch after concurrent fixture insertion with successful planned writes. The record also strongly preserves three passing controls: safe fresh retry in the fixture, no repeat mutation in a completed invocation, and no out-of-zone dispatch in the fixture.

The original historical report classifies OBS-02 as the new final-state verification gap. This edition maintains that distinction. Neither observation is sufficient to call the situation a GoDaddy production security vulnerability.

Publication verification: the archive and internal listed files passed SHA-256 checks; the native and sidecar observation pairs matched byte-for-byte; the CodeCraft PDF visual/text comparison passed; and this book was rendered and checked as a separate public artifact. These checks do not authenticate authorship of historical results or replace new independent execution.

Criterion	Final status
Public technical conclusion	Bounded consistency and final-state control finding
Production impact	Unknown / untested
New native replay execution	Not performed
Original evidence custody	Preserved
Release state	CONDITIONAL REVIEW — owner publication decision